ROLE RADAR

Job Listings Aggregator & REST API

Python | Flask | SQLite/MySQL | BeautifulSoup | APScheduler | REST API

What it does	Why it matters
Collects job listings from permitted public feeds, removes duplicates, stores structured roles, and displays them in a responsive web dashboard.	Demonstrates backend APIs, databases, scheduled automation, web scraping, deployment readiness, and clean user-facing design in one portfolio project.

Resume - 2 lines

Built a full-stack job listings platform using Python and Flask, aggregating and deduplicating public job feeds with BeautifulSoup and a scheduled APScheduler import workflow.

Designed REST APIs with JSON responses, filtering by keyword, location, and skills; integrated SQLAlchemy with SQLite/MySQL configuration and deployed a responsive live dashboard.

Portfolio links

GitHub: https://github.com/Sampreeth-AI/job-listings-aggregator-rest-api
Live demo: paste your Render public URL here after deployment.

How the project works

1. Sources	We Work Remotely RSS feeds and Remote OK public feed provide job records. Only permitted public sources are configured.
2. Scraper	BeautifulSoup parses RSS entries. The Remote OK adapter reads its public feed. Skills are extracted from titles, descriptions, and source tags.
3. Database	SQLAlchemy stores title, company, location, skills, URL, source, and timestamps. The job URL is unique, so imports update existing records instead of duplicating them.
4. REST API	GET /api/v1/jobs supports search, location, source, skills, page, and per_page. POST adds jobs; DELETE and manual scrape require X-API-Key.
5. Dashboard	The RoleRadar homepage calls the API with JavaScript, renders job cards, searches roles, and lets visitors add a job.
6. Scheduler	APScheduler triggers the import once each day at SCRAPE_HOUR_UTC. Default: 02:00 UTC / 7:30 AM IST.

Key endpoints

```
GET /api/v1/health - health check
GET /api/v1/jobs?search=Python&location=Remote&skills=flask - filtered job list
POST /api/v1/jobs - create a listing
POST /api/v1/scrape - protected manual import
DELETE /api/v1/jobs/<id> - protected removal
```

How to run locally

1. Open the project folder in VS Code. 2. Activate .venv. 3. Run `.\.venv\Scripts\python.exe run.py`. 4. Open `http://127.0.0.1:5000`.

Interview preparation

Q: How do you prevent duplicate listings?

A: Each job URL has a unique database constraint. During an import, the app first looks up the URL. It updates an existing record or inserts a new record, which is an upsert-style workflow.

Q: Why use APScheduler?

A: It runs scheduled Python jobs inside the application. Here it triggers the daily import automatically, so listings remain current without manually pressing a button.

Q: How does filtering work?

A: Flask reads query parameters. SQLAlchemy builds case-insensitive database queries across title, company, description, location, source, and skills. The API returns paginated JSON.

Q: Why SQLite and MySQL?

A: SQLite makes local setup easy. SQLAlchemy uses a DATABASE_URL, so production can switch to MySQL without rewriting the models or API queries.

Q: What would you improve for production?

A: Use a managed MySQL database, protect all write operations with authentication, use database migrations, move scraping to a dedicated worker, log failures, and respect each source's terms and rate limits.

Final GitHub checklist

1. In VS Code terminal run `git status`.
2. Run `git push`.
3. Open the GitHub repository and confirm the latest commit is visible.
4. Add the real Render link to README.md, then run `git add README.md`, `git commit -m "Add live demo link"`, and `git push`.
5. Put both GitHub and Render links on your resume.